

Task 1

- dependent and independent variables in the experiment.
 - Independent:
 - the defect detection technique(testing static, inspection)
 - The program to find defects in.
 - Dependent:
 - Total time Spent
 - Defects found
 - Defects Isolated
 - Detection Efficiency
 - Isolation Efficiency
 - Detection Frequency per defect
 - Isolation Frequency per fault
- Treatments
 - Usage-based inspection
 - Coverage-based unit testing
- Hypotheses
 - Base: Inspection and testing perform better for different purposes.
 - Null hypotheses:
 - $H_{eff-det,0}: DEFD, Insp = DEFD, Test$
 - No difference in detection effectiveness (coverage) for the different detection techniques (inspection and testing)
 - $H_{rate-det,0}: EFFD, Insp = EFFD, Test$
 - No difference in detection rate for the different detection techniques (inspection and testing)
 - $H_{defect-det,0}: FNDD, Insp = FNDD, Test$
 - No difference in distribution of the detected defects for the different detection techniques (inspection and testing)
 - $H_{eff-id,0}: DEFI, Insp = DEFI, Test$
 - No difference in isolation effectiveness for the different detection techniques (inspection and testing)
 - $H_{rate-id,0}: EFFI, Insp = EFFI, Test$

- No difference in isolation rates for the different detection techniques (inspection and testing)
- $H_{\text{defect-id},0}: FNDI, \text{Insp} = FNDI, \text{Test}$
- No difference in the distribution of isolated faults for the different detection techniques (inspection and testing)

Task 2

- dependent and independent variables in the experiment.
 - Independent:
 - Use of UML documentation
 - Dependent variables:
 - T: Time to perform the change excluding diagram modifications
 - T': Time to perform the change including diagram modifications
 - C: Number of submissions of a solution with a fault
 - C': Number of submissions of a solution with a fault, where the fault broke the existing functionality.
 - C'': Number of submissions of a solution with a fault, where the fault stemmed from not taking into account all existing behavior C''.
 - Q: Design quality in terms of following proper OO design principles.
- Treatments
 - UML
 - No UML
- Hypothesis (null hypothesis)
 - $T(\text{UML}) \geq T(\text{no-UML})$
 - Using UML does not reduce the time spent, excluding time spent on diagram updates.
 - $T'(\text{UML}) = T'(\text{no-UML})$
 - Using UML has no effect on the total time spent, including time spent on diagram updates.
 - $C(\text{UML}) \geq C(\text{no-UML})$
 - Using UML does not reduce the number of submissions of a solution with a fault.
 - $C'(\text{UML}) \geq C'(\text{no-UML})$
 - Using UML does not reduce the number of faults breaking existing functionality.
 - $C''(\text{UML}) \geq C''(\text{no-UML})$
 - Using UML does not reduce number of faults stemming from not taking into account all existing behaviours.
 - $Q(\text{UML}) \leq Q(\text{no-UML})$
 - Using UML does not improve the design quality.